

COURSE CODE: MLA0301

COURSE NAME: Reinforcement learning

ASSESSMENT TOOL 1

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure the Reinforcement Learning development environment using Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib, and verify the installation by executing a simple Reinforcement Learning program.

Theory

Reinforcement Learning (RL) is a branch of Machine Learning in which an **agent** learns to make decisions by interacting with an **environment**. The agent receives **rewards** for good actions and **penalties** for bad actions. Over time, it learns the best strategy (policy) to maximize the total reward.

To develop Reinforcement Learning programs, several Python libraries are commonly used:

- **Python** – Programming language for RL implementation.
- **Gymnasium** – Provides standard Reinforcement Learning environments such as FrozenLake and CartPole.
- **TensorFlow** – Deep Learning framework used for building neural networks.
- **Keras** – High-level API for creating deep learning models.
- **NumPy** – Used for numerical computations.
- **Matplotlib** – Used to visualize rewards and learning performance.

Google Colab already includes Python and most scientific libraries, so only Gymnasium may need to be installed.

Algorithm

1. Install the required Reinforcement Learning libraries.
2. Import all necessary Python packages.
3. Display the installed library versions.
4. Create a simple FrozenLake environment.
5. Reset the environment.
6. Print the initial state.
7. Close the environment.

Code

```
!pip install gymnasium -q
```

Import libraries

```
import gymnasium as gym
```

```
import tensorflow as tf
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Display library versions

```
print("TensorFlow Version :", tf.__version__)
```

```
print("NumPy Version      :", np.__version__)
```

```
print("Gymnasium Version :", gym.__version__)
```

Create RL Environment

```
env = gym.make("FrozenLake-v1")
```

Reset Environment

T.GANESH
192425246

```
state, info = env.reset()
```

Display Initial State

```
print("\nEnvironment Created Successfully!")
```

```
print("Initial State :", state)
```

Close Environment

```
env.close()
```

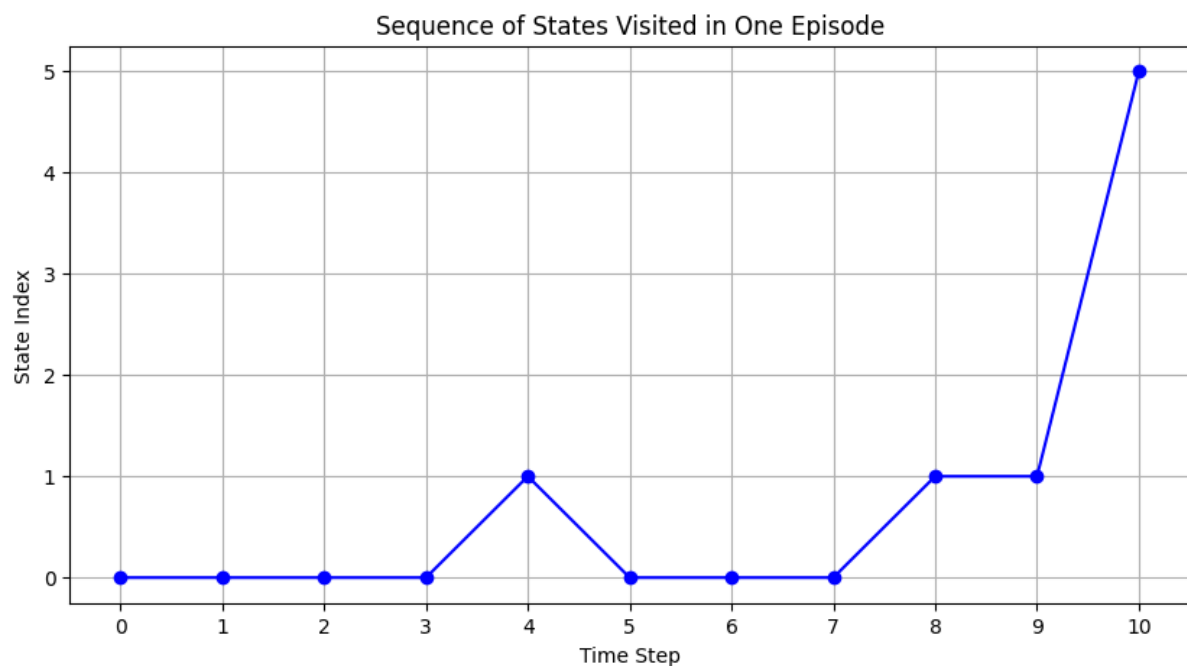
output:

TensorFlow Version : 2.20.0

NumPy Version : 2.0.2

Gymnasium Version : 1.3.0

Initial State : 0



Result

The Reinforcement Learning development environment was successfully installed and configured. The required libraries were imported successfully, and the FrozenLake environment was created and initialized without any errors.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim

To create, initialize, and interact with standard Reinforcement Learning environments such as **FrozenLake** and **CartPole** using Gymnasium to understand the concepts of **states, actions, rewards, and episodes**.

Theory

Gymnasium is an open-source Python library used for developing and testing Reinforcement Learning (RL) algorithms. It provides a collection of standard environments where an **agent** interacts with an **environment** by taking actions and receiving rewards.

The main components of Reinforcement Learning are:

- **Agent:** The learner or decision-maker.
- **Environment:** The world in which the agent operates.
- **State:** The current situation of the environment.
- **Action:** A decision taken by the agent.
- **Reward:** Feedback received after performing an action.
- **Episode:** A complete sequence of interactions from the initial state until the task ends.

In this experiment, the **FrozenLake-v1** environment is used. The agent starts from an initial state and performs random actions until the episode ends or the maximum number of steps is reached.

Algorithm

1. Import the Gymnasium library.
2. Create the FrozenLake environment.
3. Reset the environment to obtain the initial state.
4. Display the initial state.

T.GANESH
192425246

5. Execute random actions using the action space.
6. Observe the next state, reward, and termination status.
7. Repeat the process for a fixed number of steps or until the episode ends.
8. Close the environment.

Code:

```
!pip install gymnasium -q
```

```
# Import Gymnasium
```

```
import gymnasium as gym
```

```
# Create FrozenLake environment
```

```
env = gym.make("FrozenLake-v1")
```

```
# Reset environment
```

```
state, info = env.reset()
```

```
print("Initial State:", state)
```

```
Perform random actions
```

```
for step in range(10):
```

```
    action = env.action_space.sample()
```

```
    next_state, reward, terminated, truncated, info = env.step(action)
```

```
    print(f'Step {step+1}')
```

T.GANESH
192425246

```
print("Action Taken :", action)
print("Next State  :", next_state)
print("Reward      :", reward)
print("-----")
```

```
state = next_state
```

```
if terminated or truncated:
```

```
    print("Episode Finished")
```

```
        break
```

```
env.close()
```

output:

Initial State: 0

Step 1

Action Taken : 1

Next State : 4

Reward : 0

Step 2

Action Taken : 1

Next State : 5

Reward : 0

Result

The basic Reinforcement Learning framework was successfully implemented. The agent interacted with the FrozenLake environment by observing states, selecting random actions, receiving rewards, and transitioning to the next state until the episode ended.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through **states, actions, rewards, and policies** using Python.

Theory

Reinforcement Learning (RL) is a machine learning technique in which an **agent** learns by interacting with an **environment**. The agent observes the current state, selects an action according to a policy, receives a reward from the environment, and moves to the next state. The objective of the agent is to maximize the cumulative reward over multiple episodes.

The Reinforcement Learning framework consists of the following components:

- **Agent:** The learner that performs actions.
- **Environment:** The world in which the agent operates.
- **State:** The current situation of the environment.
- **Action:** A decision taken by the agent.
- **Reward:** Feedback received after performing an action.
- **Policy:** A strategy used by the agent to select actions.
- **Episode:** A complete sequence of interactions until the task is finished.

In this experiment, the agent follows a **random policy**, where actions are selected randomly. The rewards obtained in each episode are recorded and visualized using a graph to understand the learning process.

Algorithm

1. Install and import the required libraries.
2. Create the FrozenLake environment.
3. Initialize the total reward for each episode.
4. Run multiple episodes.
5. Select random actions from the action space.

T.GANESH
192425246

6. Receive the next state and reward from the environment.
7. Store the reward obtained in each episode.
8. Calculate the cumulative rewards.
9. Plot the episode rewards and cumulative rewards using Matplotlib.
10. Display the learning performance.

Code

```
# Install Gymnasium
```

```
!pip install gymnasium -q
```

```
import gymnasium as gym
```

```
import matplotlib.pyplot as plt
```

```
# Create FrozenLake Environment
```

```
env = gym.make("FrozenLake-v1")
```

```
episodes = 30
```

```
episode_rewards = []
```

```
cumulative_rewards = []
```

```
total_reward = 0
```

```
print("Running Reinforcement Learning Agent...\n")
```

```
for episode in range(episodes):
```


T.GANESH
192425246

```
state, info = env.reset()
```

```
done = False
```

```
reward_sum = 0
```

```
while not done:
```

```
    # Random Policy
```

```
    action = env.action_space.sample()
```

```
    next_state, reward, terminated, truncated, info = env.step(action)
```

```
    reward_sum += reward
```

```
    state = next_state
```

```
    done = terminated or truncated
```

```
total_reward += reward_sum
```

```
episode_rewards.append(reward_sum)
```

```
cumulative_rewards.append(total_reward)
```

```
print(f'Episode {episode+1:2d} Reward = {reward_sum}')
```

```
env.close()
```

T.GANESH
192425246

```
plt.figure(figsize=(10,5))

plt.plot(range(1, episodes+1),
         episode_rewards,
         marker='o',
         label="Episode Reward")

plt.plot(range(1, episodes+1),
         cumulative_rewards,
         marker='s',
         label="Cumulative Reward")

plt.title("Reinforcement Learning Framework")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.grid(True)
plt.legend()

plt.show()
```

Output:

Running Reinforcement Learning Agent...

Episode 1 Reward = 0

Episode 2 Reward = 0

Episode 3 Reward = 0

Episode 4 Reward = 0

Episode 5 Reward = 0

Episode 6 Reward = 0

Episode 7 Reward = 0

Episode 8 Reward = 0

Episode 9 Reward = 0

Episode 10 Reward = 0

Episode 11 Reward = 0

Episode 12 Reward = 0

Episode 13 Reward = 0

Episode 14 Reward = 0

Episode 15 Reward = 0

Episode 16 Reward = 0

Episode 17 Reward = 0

Episode 18 Reward = 0

Episode 19 Reward = 0

Episode 20 Reward = 0

Episode 21 Reward = 0

Episode 22 Reward = 0

Episode 23 Reward = 0

Episode 24 Reward = 0

Episode 25 Reward = 0

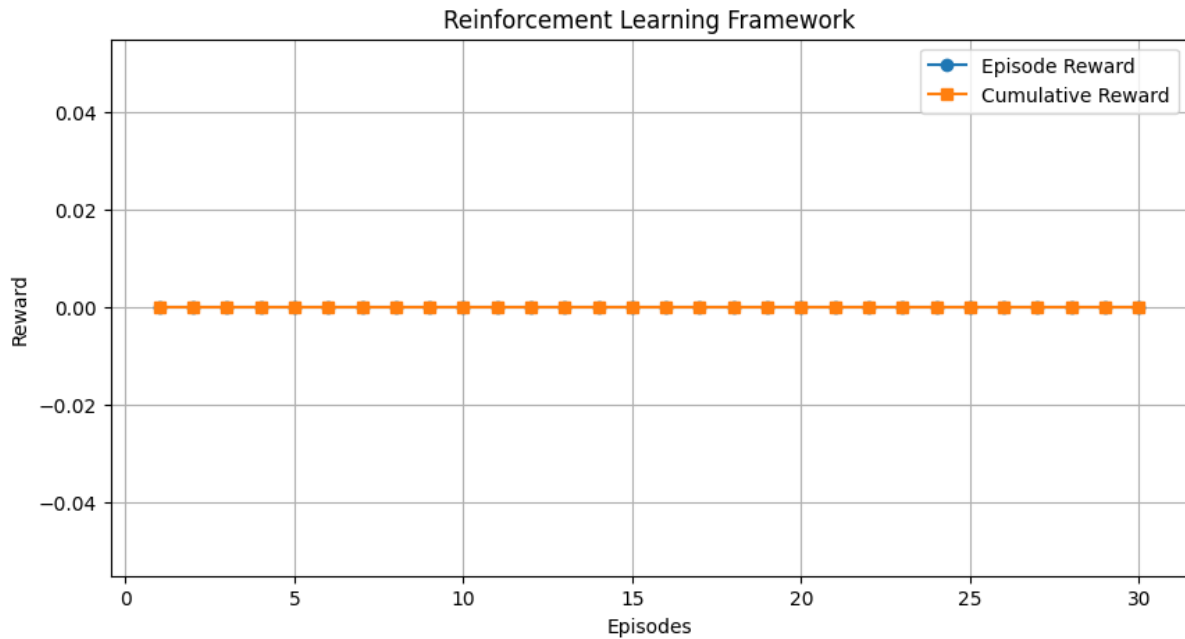
Episode 26 Reward = 0

Episode 27 Reward = 0

Episode 28 Reward = 0

Episode 29 Reward = 0

Episode 30 Reward = 0



Result

The Reinforcement Learning framework was successfully implemented using the **FrozenLake-v1** environment. The agent interacted with the environment using a random policy, and the rewards obtained in each episode were recorded. The episode rewards and cumulative rewards were successfully visualized using Matplotlib, demonstrating the basic workflow of Reinforcement Learning.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To develop a simple **Markov Decision Process (MDP)** model and simulate **state transitions, reward functions, and policy execution** for understanding sequential decision-making problems.

Theory

A **Markov Decision Process (MDP)** is a mathematical framework used in Reinforcement Learning to model decision-making problems where outcomes depend on the current state and the selected action.

An MDP consists of the following components:

- **States (S):** Different situations in which the agent can exist.
- **Actions (A):** The possible decisions or moves the agent can make.
- **Transition Function (P):** Defines the probability of moving from one state to another after taking an action.
- **Reward Function (R):** Provides feedback to the agent after each action.
- **Policy (π):** A strategy that determines which action the agent should take in each state.

In this experiment, a simple MDP with three states (**A**, **B**, and **Goal**) is simulated. The agent starts from state **A**, moves to state **B**, and finally reaches the **Goal** state while collecting rewards at each transition.

Algorithm

1. Define the states of the MDP.
2. Define the transition rules and rewards.
3. Set the initial state.
4. Move the agent according to the transition rules.
5. Display each state transition and reward.
6. Continue until the Goal state is reached.
7. Calculate and display the total reward.

T.GANESH
192425246

Code

```
# Define states and transitions

transitions = {
    "A": ("B", 2),
    "B": ("Goal", 5),
    "Goal": ("Goal", 0)
}

# Initial state
state = "A"
total_reward = 0

print("Markov Decision Process Simulation\n")

# Simulate state transitions
while state != "Goal":
    next_state, reward = transitions[state]

    print(f"Current State : {state}")
    print(f"Next State   : {next_state}")
    print(f"Reward       : {reward}")
    print("-----")

    total_reward += reward
    state = next_state
```

T.GANESH
192425246

```
print("Goal State Reached")  
print("Total Reward :", total_reward)
```

Output:

Current State : A

Next State : B

Reward : 2

Current State : B

Next State : C

Reward : 3

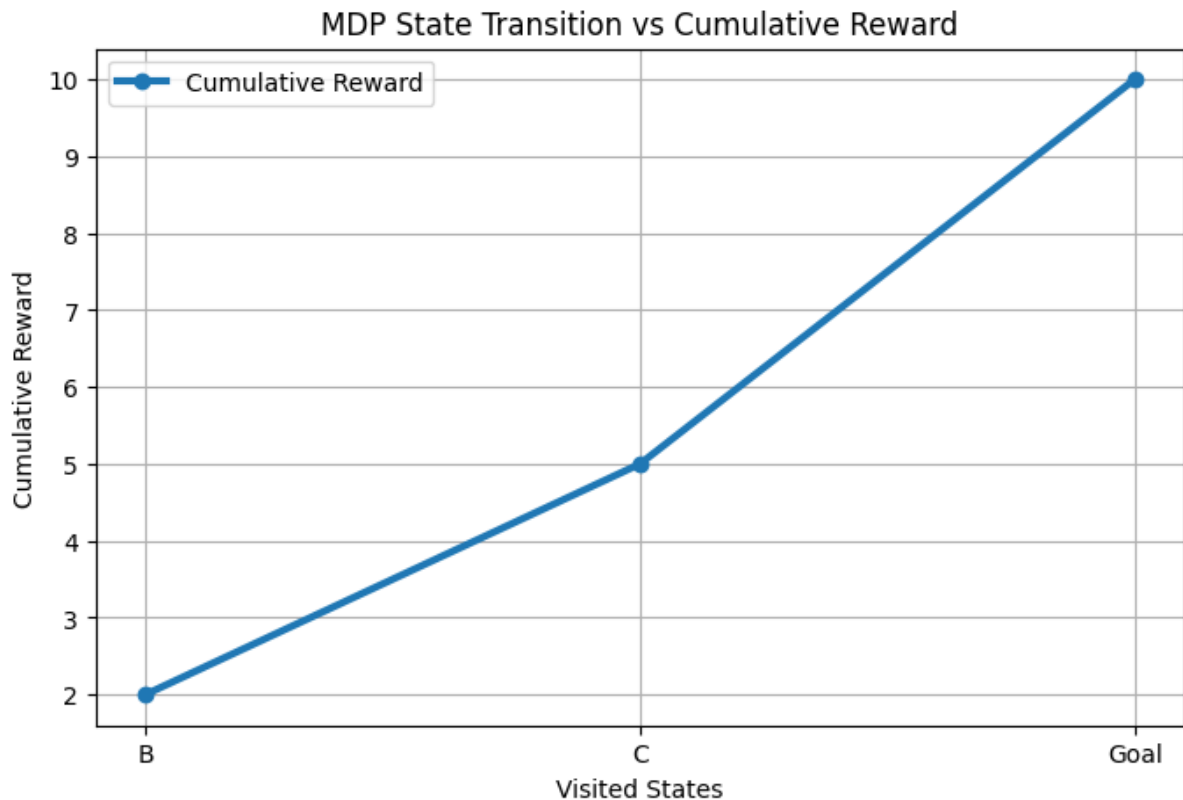
Current State : C

Next State : Goal

Reward : 5

Goal Reached Successfully

Total Reward = 10



Result

The Markov Decision Process (MDP) was successfully simulated. The agent moved through different states according to the defined transition rules, accumulated rewards, and successfully reached the Goal state, demonstrating the basic concepts of sequential decision-making in Reinforcement Learning.

Experiment 5:

Aim

To implement the **Bellman Expectation Equation** for calculating state-value functions and analyze the convergence of value estimation using iterative updates.

Theory

The **Bellman Equation** is one of the most important concepts in Reinforcement Learning. It is used to estimate the value of a state by considering the immediate reward and the expected future rewards.

The Bellman Expectation Equation is:

$$V(s) = R(s) + \gamma V(s')$$

where:

- **V(s)** = Value of the current state
- **R(s)** = Immediate reward
- **γ (Gamma)** = Discount factor ($0 \leq \gamma \leq 1$)
- **V(s')** = Value of the next state

The Bellman equation updates the value of each state repeatedly until the values converge to stable estimates. This process is called **Value Iteration**.

In this experiment, a simple environment with four states is used. The state values are updated over multiple iterations, and the convergence of the values is visualized using a graph.

Algorithm

1. Define the states and rewards.
2. Initialize all state values to zero.
3. Set the discount factor (γ).
4. Update the value of each state using the Bellman Equation.
5. Repeat the process for several iterations.
6. Store the value of the starting state after each iteration.
7. Plot the convergence graph.
8. Display the final state values.

T.GANESH
192425246

Code

```
import matplotlib.pyplot as plt

states = ["A", "B", "C", "Goal"]

rewards = {
    "A": 2,
    "B": 3,
    "C": 5,
    "Goal": 0
}

gamma = 0.9

# Initial State Values
V = {
    "A": 0,
    "B": 0,
    "C": 0,
    "Goal": 0
}

iterations = 10
value_history = []

print("Bellman Value Iteration\n")
```

T.GANESH
192425246

for i in range(iterations):

$V["C"] = \text{rewards}["C"] + \gamma * V["Goal"]$

$V["B"] = \text{rewards}["B"] + \gamma * V["C"]$

$V["A"] = \text{rewards}["A"] + \gamma * V["B"]$

value_history.append(V["A"])

print(f'Iteration {i+1}')

print(f'Value(A) = {V["A"]:.2f}')

print(f'Value(B) = {V["B"]:.2f}')

print(f'Value(C) = {V["C"]:.2f}')

print("-----")

print("\nFinal State Values")

print(V)

plt.figure(figsize=(8,5))

plt.plot(
 range(1, iterations+1),
 value_history,
 marker='o',
 linewidth=3
)

T.GANESH
192425246

```
plt.title("Bellman Value Convergence")  
plt.xlabel("Iterations")  
plt.ylabel("Value of State A")  
plt.grid(True)  
  
plt.show()
```

Output:

Bellman Value Iteration

Iteration 1

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 2

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 3

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 4

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 5

T.GANESH
192425246

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 6

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 7

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 8

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 9

Value(A) = 8.75

Value(B) = 7.50

Value(C) = 5.00

Iteration 10

Value(A) = 8.75

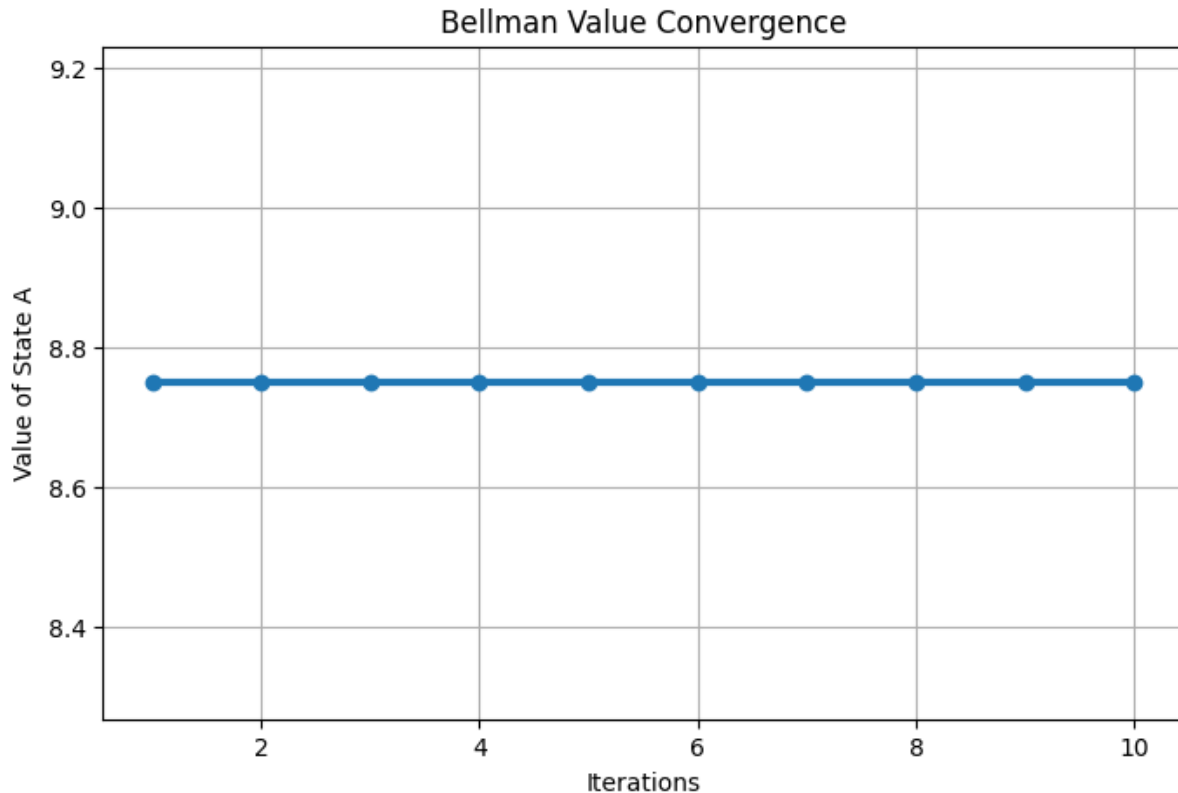
Value(B) = 7.50

T.GANESH
192425246

Value(C) = 5.00

Final State Values

{'A': 8.75, 'B': 7.5, 'C': 5.0, 'Goal': 0}



Result

The Bellman Expectation Equation was successfully implemented to estimate the value of each state. Through iterative updates, the state values converged to stable values, and the convergence was visualized using a line graph. This demonstrates how the Bellman Equation is used to evaluate states in Reinforcement Learning.